- この章の修得目標

第4章　シーケンス図

- シーケンス図の基本表記を理解する
- メッセージの種類と表記の違いを理解する
- オブジェクトの生成と消滅の表現方法を理解する
- 選択処理や繰り返し処理の表現方法を理解する

本章では、シーケンス図の基本表記および選択処理や繰り返し処理の表現方法について学びます。

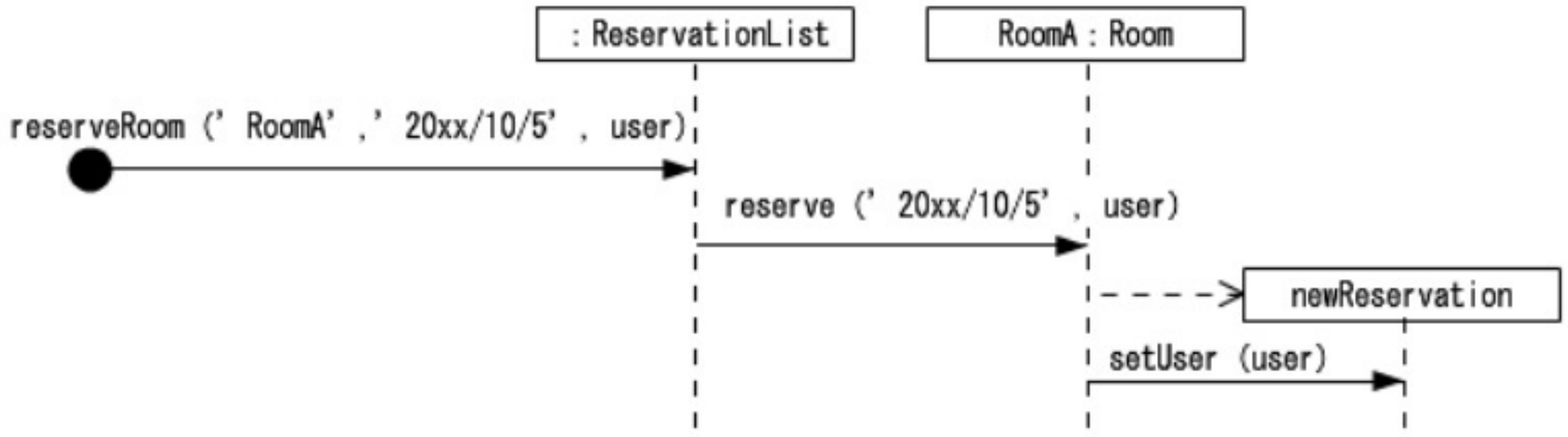
第4章　シーケンス図

基本表記

● 特徴

■ オブジェクトの相互作用を表わす
■ 縦軸は時間を示す
■ 処理の流れがわかりやすい

シーケンス図

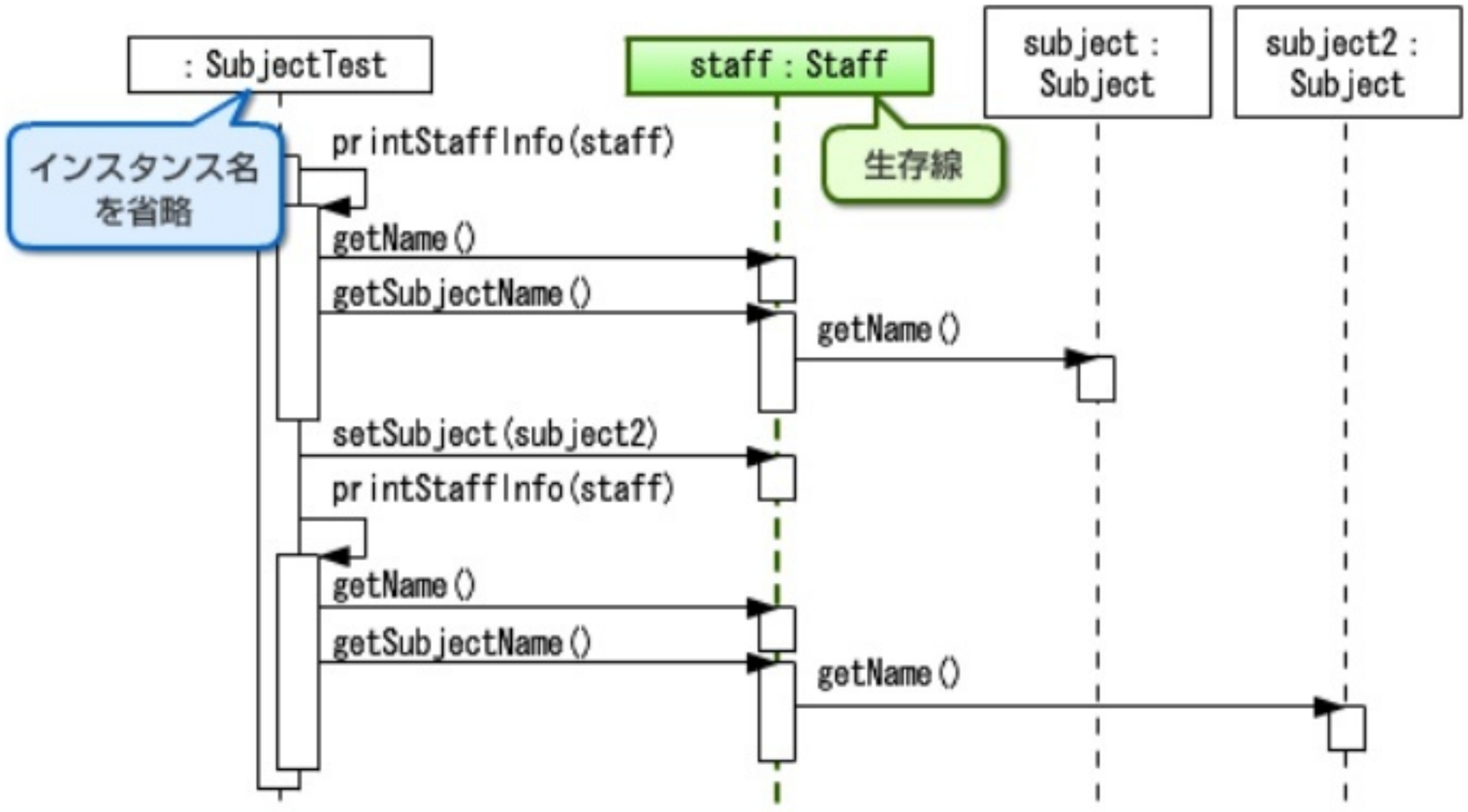


シーケンス図はオブジェクトの相互作用を表す図です。相互作用とは、複数のオブジェクトがメッセージを送信し合って動作する様子のことです。シーケンス図の縦軸は時間を表すので、図を上から読むと処理の流れがわかります。

基本表記

● 生存線

■ 一連の処理に参加するオブジェクトを表す
■ 名前を「インスタンス名：クラス名」の形式で記述する
　－インスタンス名、クラス名は省略可能

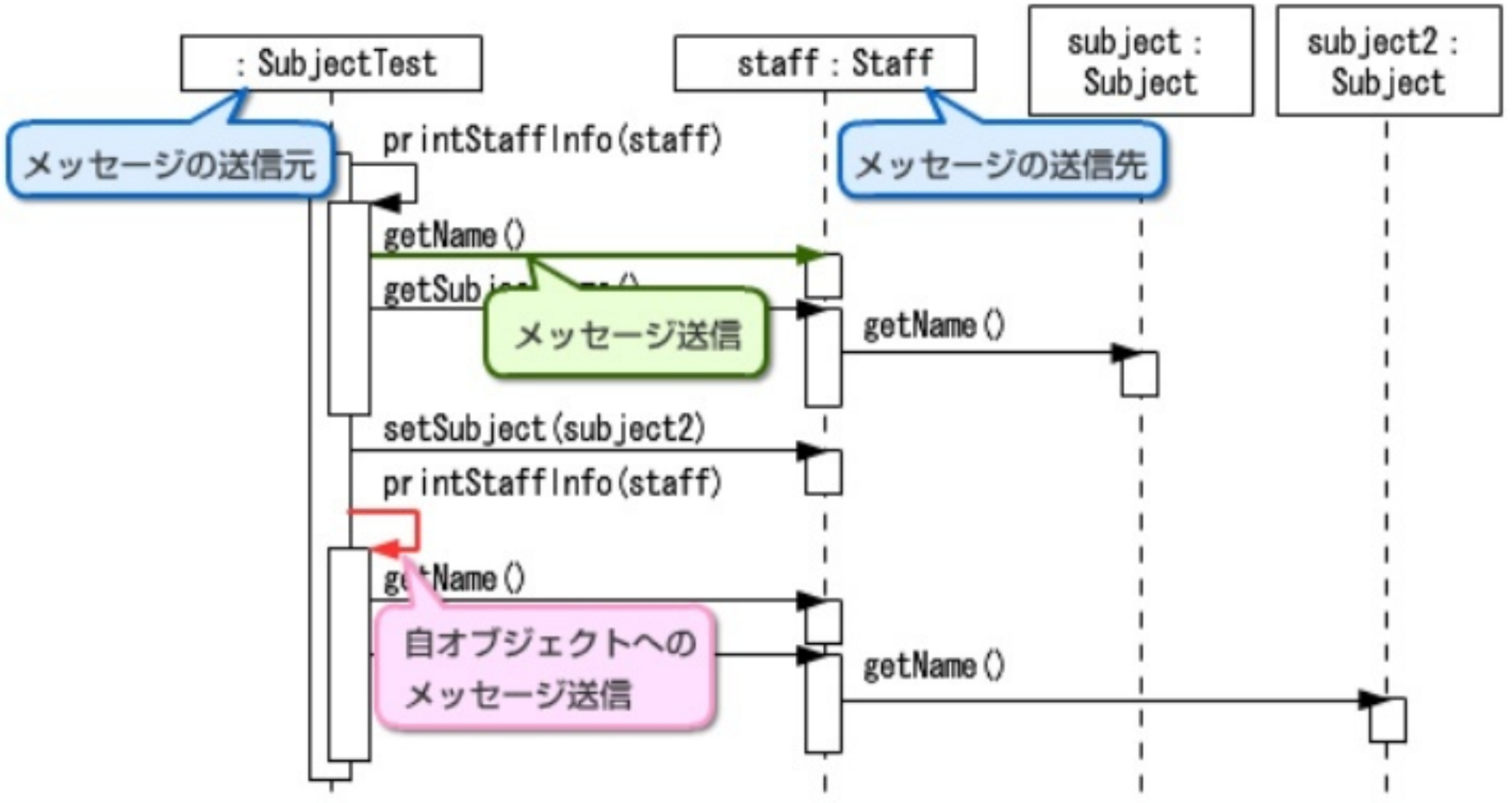


シーケンス図の上部に横に並んだ長方形とそこから下に伸びる点線を生存線といいます。生存線は一連の処理に参加するオブジェクトを表します。長方形の中には名前を「インスタンス名:クラス名」の形式で書きますが、インスタンス名およびクラス名は省略できます。

基本表記

● メッセージ送信

■ 生存線と生存線を結ぶ矢印
■ 矢印の元にあるオブジェクトが、矢印の先にあるオブジェクトにメッセージを送る
■ 自分自身の生存線に戻る矢印は、自オブジェクトに対するメッセージ送信を表す

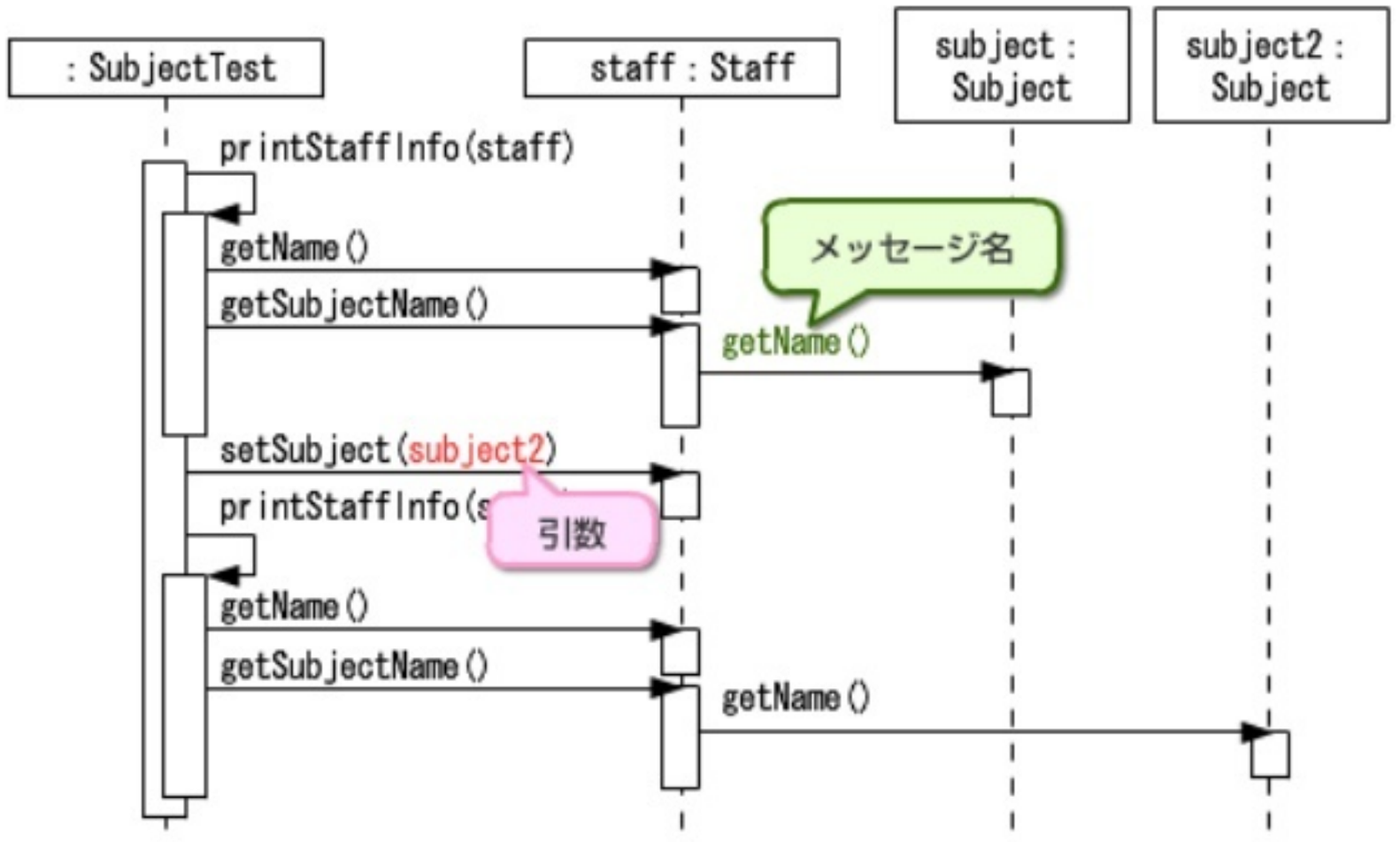


生存線と生存線の間に引いた矢印はメッセージ送信を表します。矢印の元にあるオブジェクトが先にあるオブジェクトにメッセージを送ります。
自分自身の生存線に戻る矢印は自オブジェクトに送るメッセージ(ローカルなメッセージ送信)を表します。

基本表記

● メッセージ名

■ 送信しているメッセージを表す
■ 括弧の中にメッセージ送信時に渡す引数を書ける

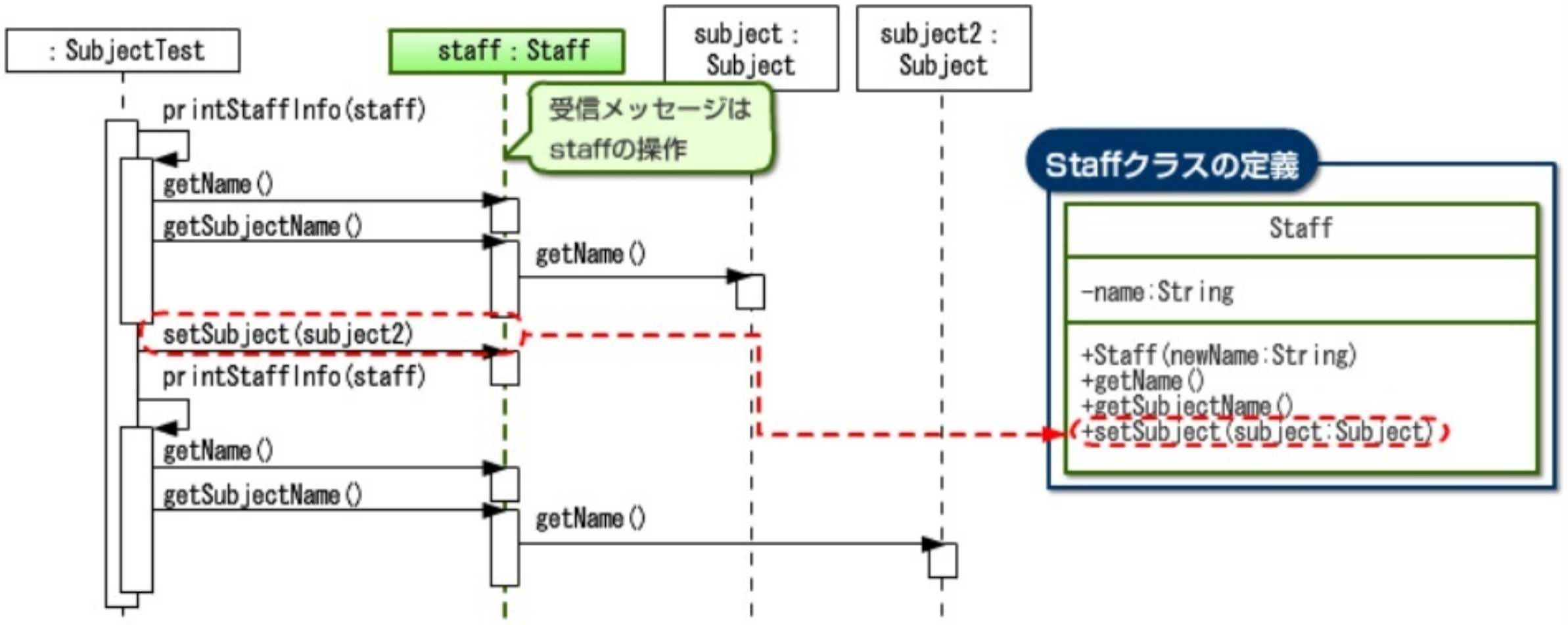


矢印の上にはメッセージ名を書き、どのようなメッセージを送信しているのかを示します。メッセージ名の後ろの括弧にはメッセージ送信時に渡す具体的な引数を書くことができます。

基本表記

● メッセージ送信と操作

■ メッセージ送信　＝　操作の呼び出し
■ 操作はメッセージを受信するオブジェクトが持つ

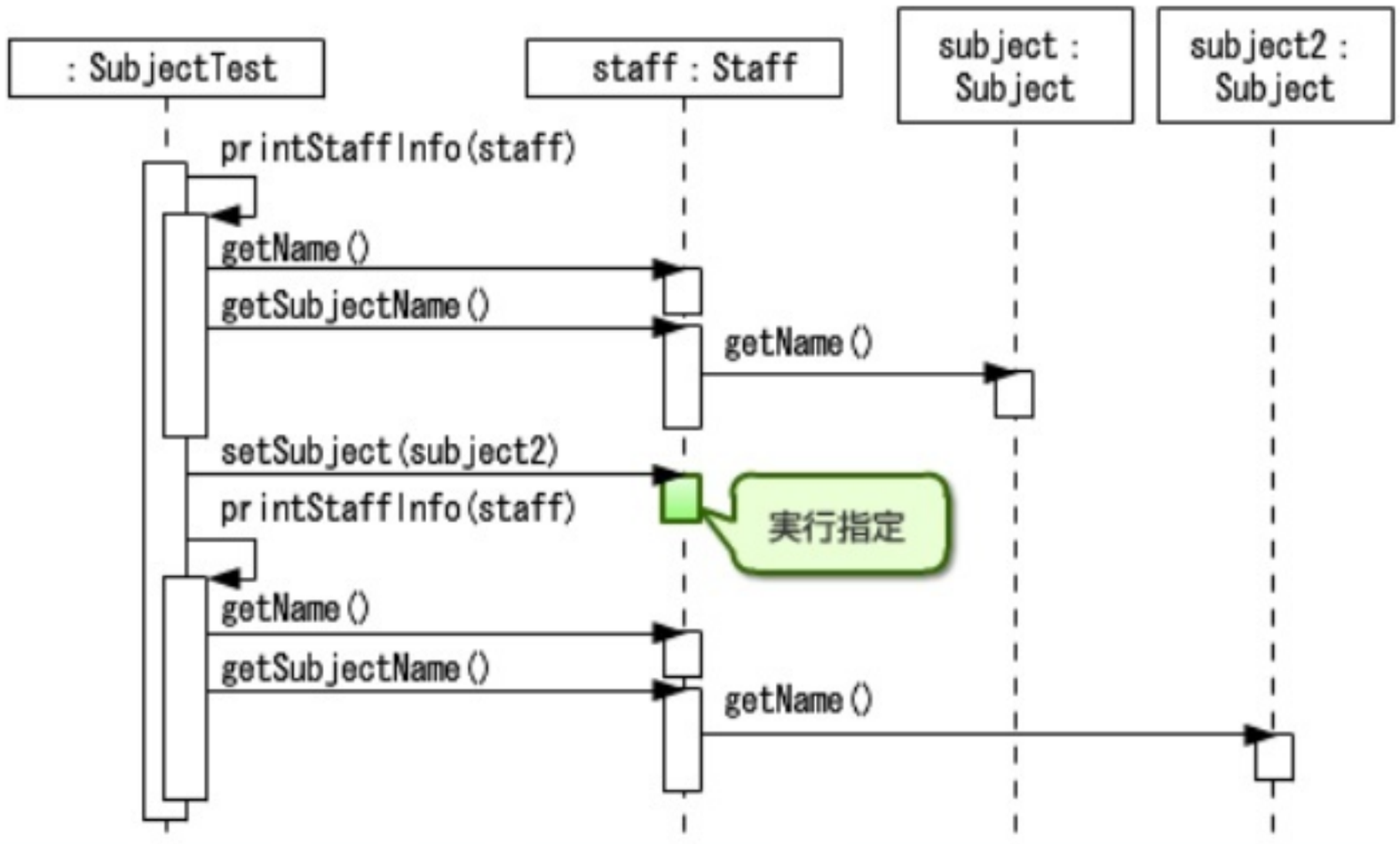


メッセージ送信は操作の呼び出しに相当し、その操作は受信するオブジェクトが持ちます。
たとえば、このシーケンス図のstaffオブジェクトを表す生存線に向かうメッセージgetNameやgetSubjectName、setSubjectはstaffオブジェクトが持つ操作です。したがって、クラス図ではstaffオブジェクトのテンプレートであるStaffクラスの操作として表記されます。

基本表記

● 実行指定

■ 生存線上の縦長の長方形
■ 呼び出された操作の実行中を表す
■ 省略可能



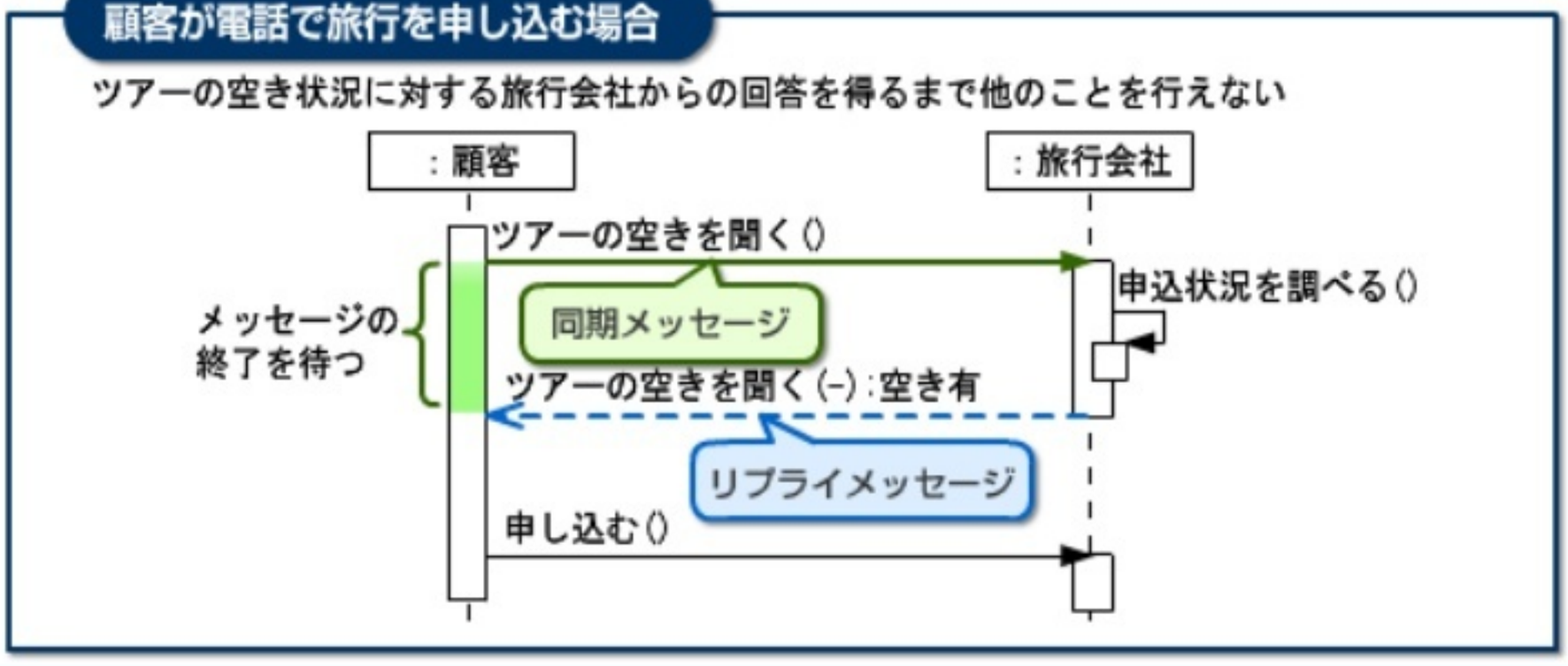
生存線上の縦長の長方形を「実行指定」といいます。実行指定はメッセージを受信したオブジェクトがその処理を実行している様子を表します。自分自身に送信したメッセージの実行は、実行指定を重ねて表します。
実行指定の記述は省略できます。

基本表記

● 同期メッセージとリプライメッセージ

■ 同期メッセージ
- －送信したメッセージの終了を待つ
- －矢先が塗りつぶしの三角形の矢印

■ リプライメッセージ
- －同期メッセージの終了を表す
- －同期メッセージと反対向きの点線矢印
- －矢印の上には「メッセージ名(-)：戻り値」の形式で戻り値を書ける



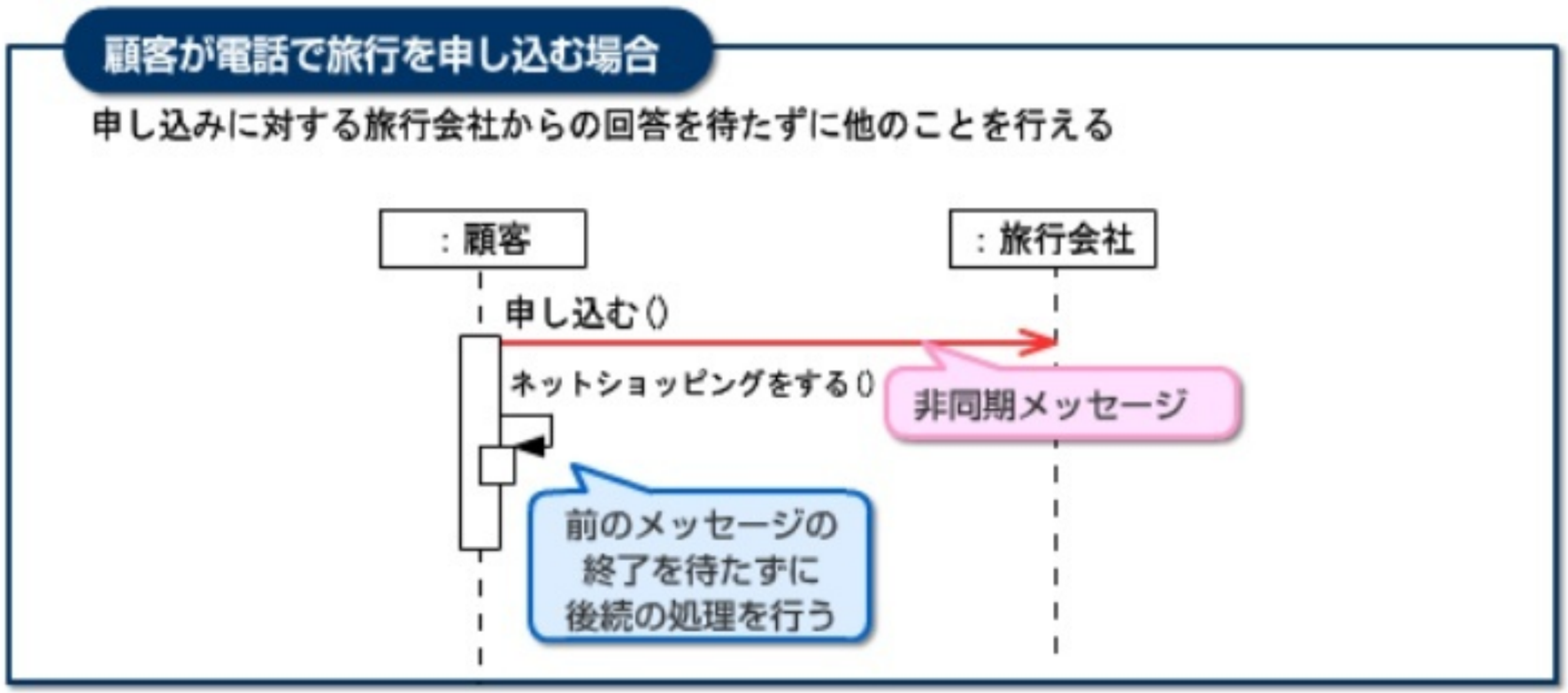
メッセージには種類があります。『同期メッセージ』は送信したメッセージの終了を待つタイプのメッセージです。メッセージの送信元オブジェクトは呼び出した操作が終了するまで次の処理を行わずに待ち続けます。同期メッセージは矢先が塗りつぶしの三角形の矢印で描きます。
同期メッセージの終了を明示的に表すには『リプライメッセージ』を使います。リプライメッセージは同期メッセージと逆向きの点線矢印で表します。矢印の上にはメッセージ名と戻り値を書くことができます。同期メッセージの終了後、呼び出し元に制御が戻ることは明らかなので、リプライメッセージは多くの場合省略します。

基本表記

● 非同期メッセージ

■ 送信したメッセージの終了を待たない

■ 矢先が開いた矢印

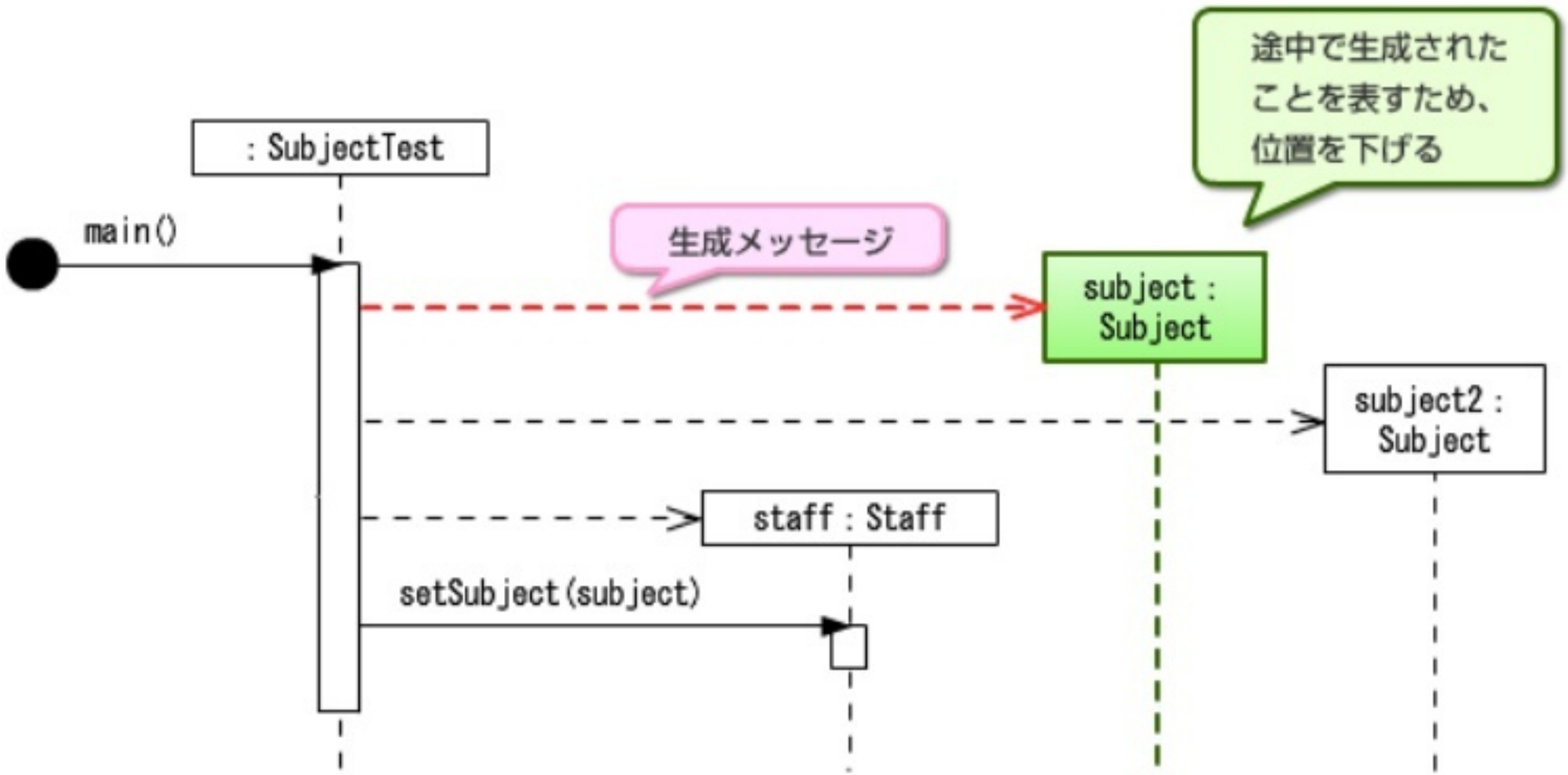


『非同期メッセージ』は送信したメッセージの終了を待たないタイプのメッセージです。メッセージの送信元オブジェクトは呼び出した操作の終了を待たずに次の処理へと進みます。非同期メッセージは矢先が開いた矢印で描きます。

基本表記

● 生成メッセージ

■ 生存線の長方形部分に向かう点線矢印
■ オブジェクトの生成を表す



『生成メッセージ』はオブジェクトの生成を表します。生存線の長方形部分に点線矢印をつなぎます。シーケンス図の縦軸は時間の経過を表すので、シーケンス図が表す一連の処理の途中で生成されるオブジェクトの生存線は位置が下にさがります。

第4章　シーケンス図

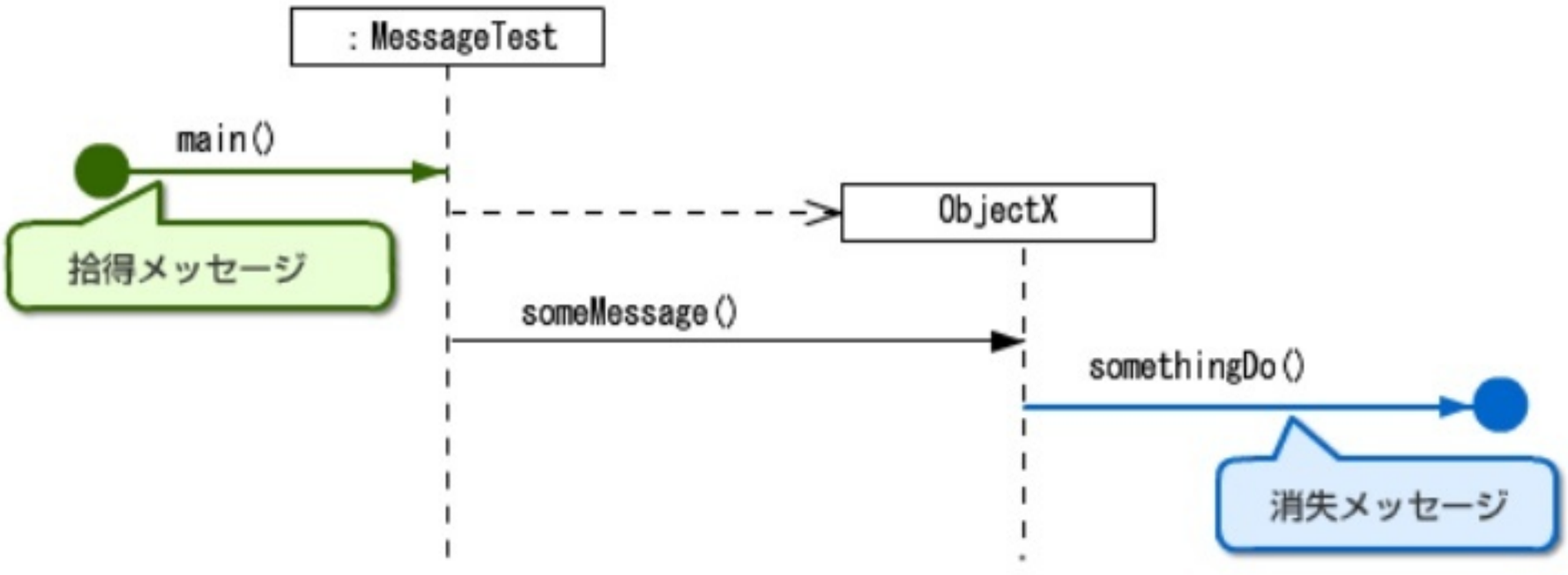
基本表記

● 拾得メッセージと消失メッセージ

■ 拾得メッセージ
　－送信元オブジェクトが不明なメッセージ

■ 消失メッセージ
　－送信先オブジェクトが不明なメッセージ

■ 不明な送信元/送信先オブジェクト
　－塗りつぶしの丸印で表す



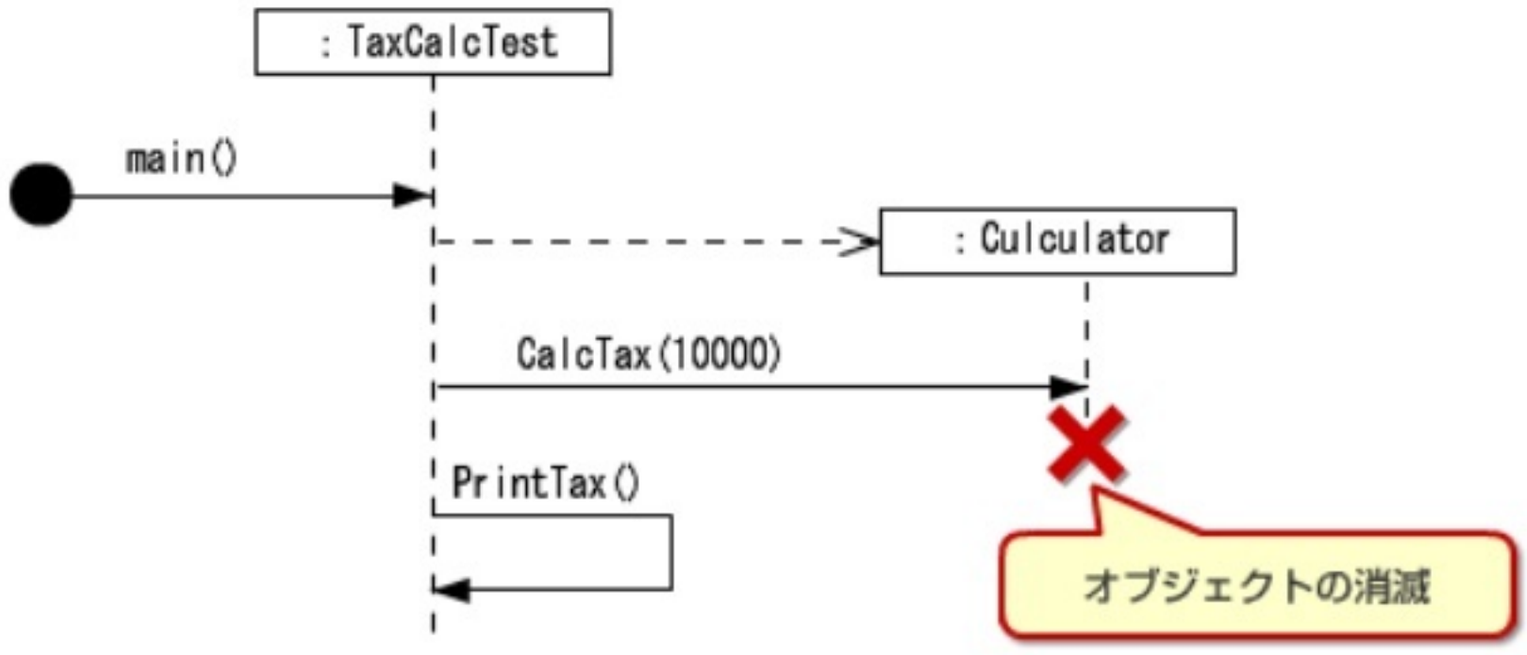
※実行指定の記述は省略

『拾得メッセージ』は送信元オブジェクトが分からないメッセージ、『消失メッセージ』は送信先オブジェクトが分からないメッセージをいいます。どちらも不明な送信元、送信先オブジェクトは塗りつぶしの丸印で表します。シーケンス図に記載していないオブジェクトとメッセージを送受信する時やメッセージの送信元や送信先を明確にする必要がない時などに使います。メッセージの形状はメッセージの種類に依存します。同期メッセージなら矢先が塗りつぶしの三角形の矢印、非同期メッセージなら矢先が開いた矢印で記述します。

基本表記

● オブジェクトの消滅

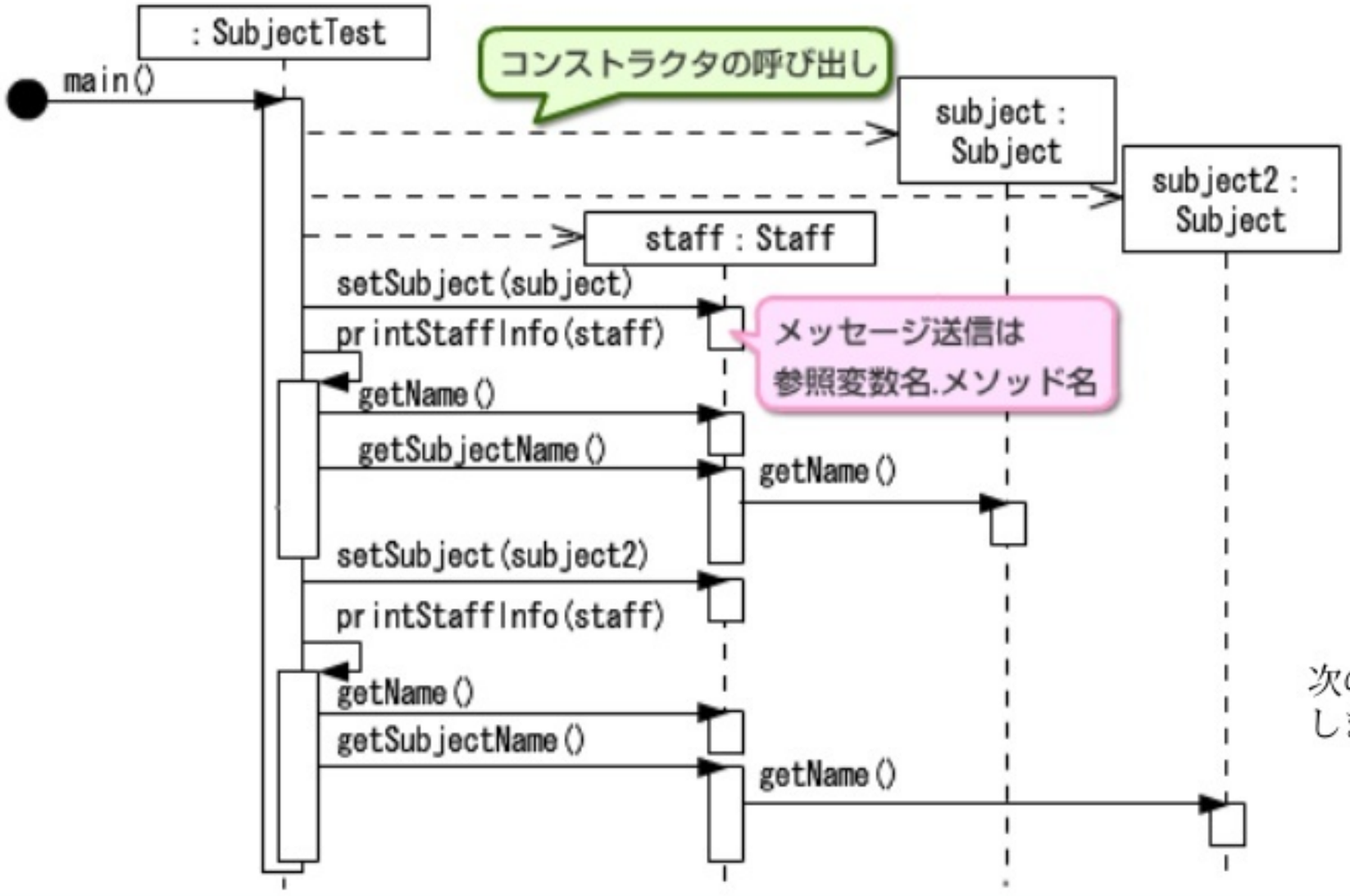
■ 生存線を切り、×印を書く



一連の処理の途中でオブジェクトが消滅することを表すには、オブジェクトの存在を示す生存線を途中で切り、大きな×印を書きます。

基本表記

● Java言語との対応



次のページにソースを表示します。

生成メッセージはコンストラクタの呼び出しに相当します。また、メッセージ送信はメッセージの送信先オブジェクトを参照する参照変数を使ったメソッド呼び出しとしてコード化されます。ただし、自分自身へのメッセージ送信には参照変数名は不要です。
「ソースを表示」ボタンをクリックし、シーケンス図とJavaコードとの対応を確認してください。

基本表記

● Java言語との対応

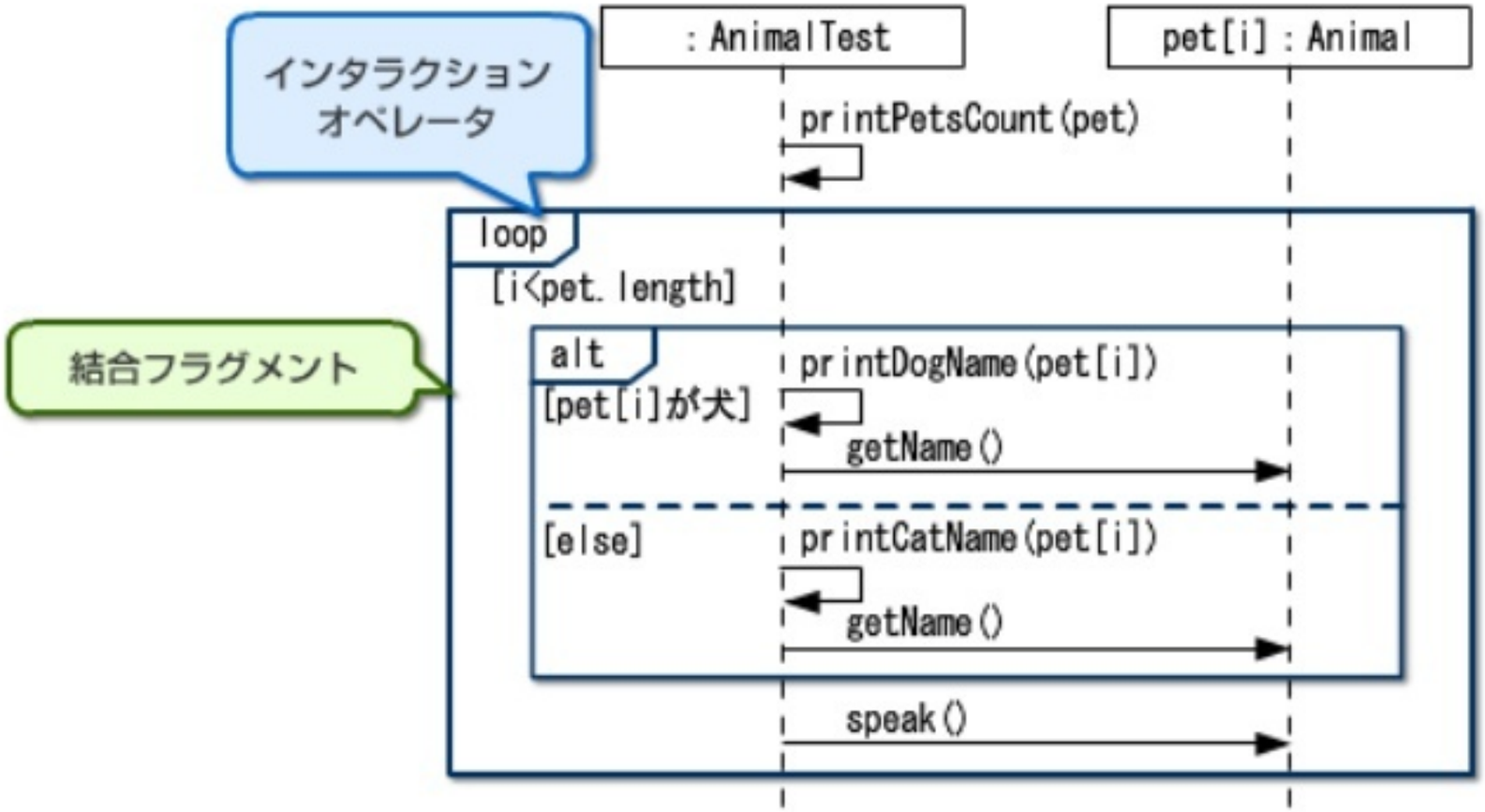
【SubjectTest.java】

```
1   class SubjectTest{
2       public static void main(String args[]){
3           // Subject オブジェクトの生成
4           Subject subject = new Subject("経営学科");
5           Subject subject2 = new Subject("自然科学科");
6
7           // Staff オブジェクトの生成と Subject オブジェクトへのリンク設定
8           Staff staff = new Staff("山田　一郎");
9            staff.setSubject(subject);
10
11          // Staff オブジェクトの利用
12          printStaffInfo(staff);
13
14          staff.setSubject(subject2);
15
16          System.out.println("=====学科変更=====");
17          printStaffInfo(staff);
18      }
19
20      static void printStaffInfo(Staff staff){
21          System.out.println("名前：" + staff.getName());
22          System.out.println("所属：" + staff.getSubjectName());
23          System.out.println();
24      }
25  }
26
27  class Subject{
28      // 属性(メンバ変数)の定義
29      private String name;
30
31      // コンストラクタ
32      public Subject(String newName){
33          name = newName;
34      }
35
36      // 操作(メソッド)の定義
37      public String getName(){
38          return  name;
39      }
40  }
41
42  class Staff{
43      // 属性(メンバ変数)の定義
44      private String name;
45
46      // 関連(メンバ変数)の定義
47      private Subject mySubject;
48
49      // コンストラクタ
50      public Staff(String newName){
51          name = newName;
52      }
53
54      // 操作(メソッド)の定義
55      public String getName(){
56          return  name;
57      }
58      public void setSubject(Subject Subject){
59          mySubject = Subject;
60      }
61      public String getSubjectName(){
62          return mySubject.getName();
63      }
64  }
65
```

結合フラグメント

● 結合フラグメントとは

■ メッセージを囲む長方形
■ 選択処理や繰り返し処理を記述する時に使う
■ 処理の種類はインタラクションオペレータで示す

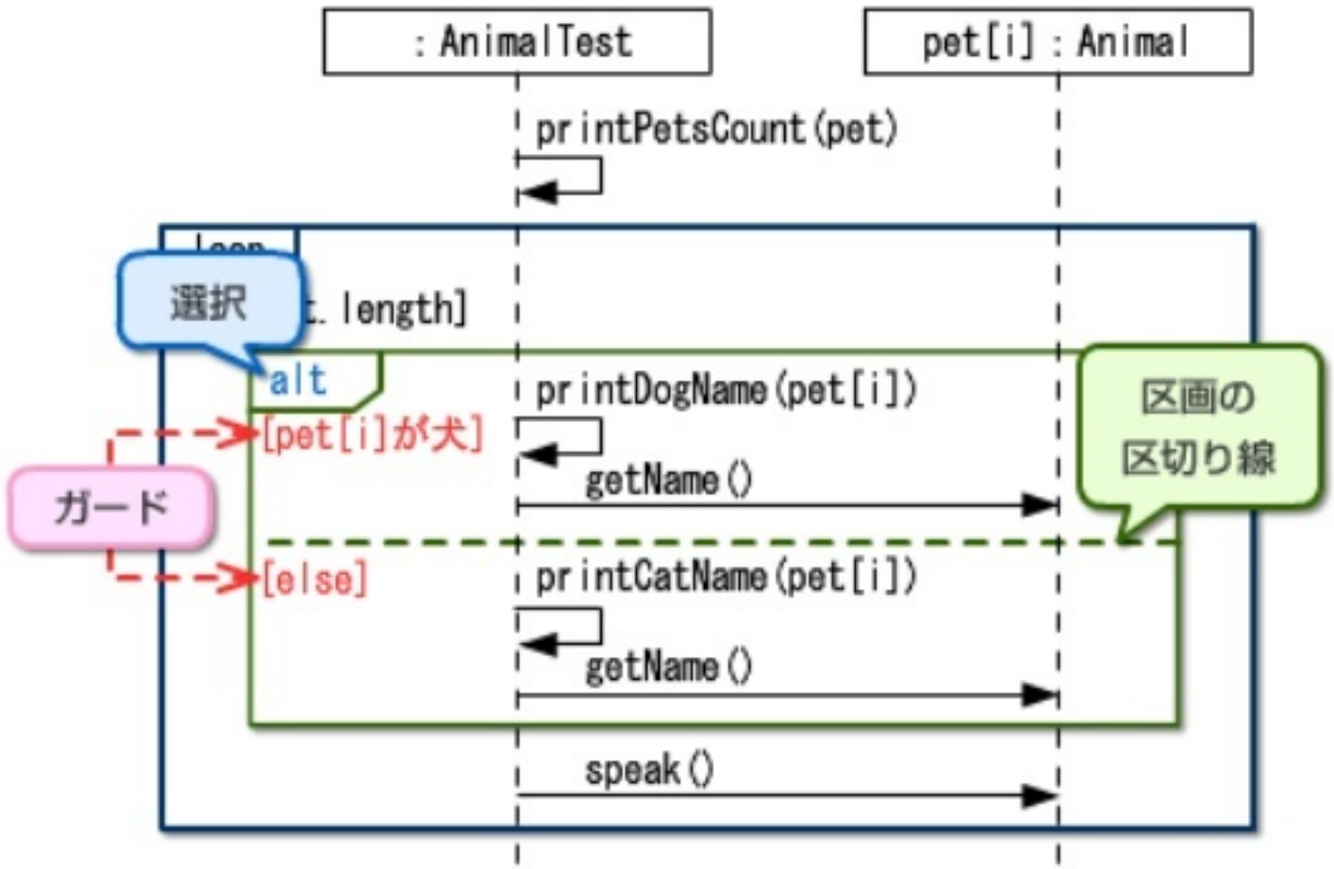


メッセージを囲む長方形を「結合フラグメント」といいます。結合フラグメントは選択処理や繰り返し処理などを示す時に使います。左上の五角形の部分に処理の種類を表すインタラクションオペレータを書きます。ここではよく使うインタラクションオペレータを説明します。

結合フラグメント

● alt

■ 選択処理を表すインタラクションオペレータ
■ 条件（ガード）を満たす最初の区画を実行する
■ elseの区画はいずれの条件も満たさなかった場合に実行される
■ elseの区画がなく、いずれの条件も満たさなかった場合は何もしない

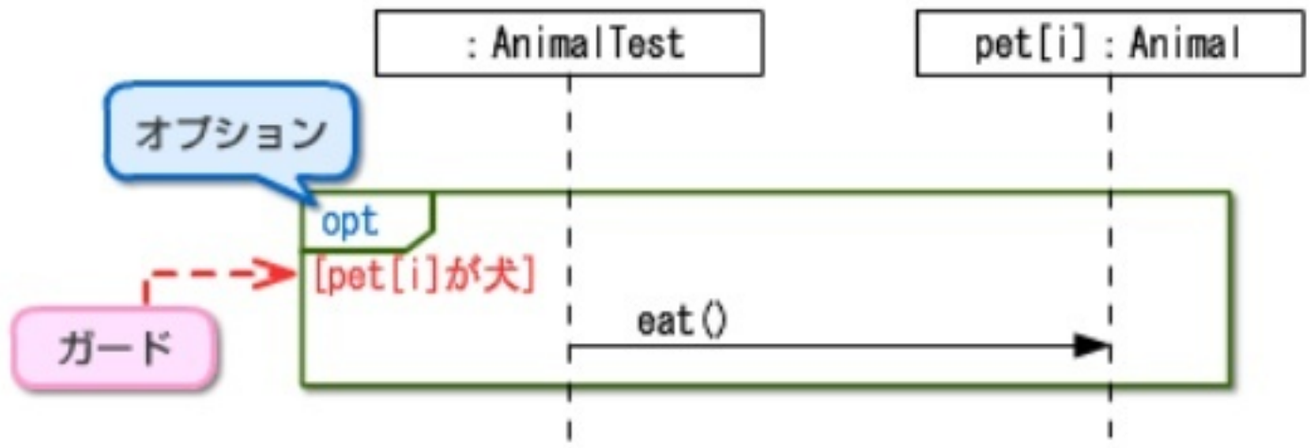


『alt』は選択処理を示すインタラクションオペレータで、条件によって処理を分岐させる場合に使用します。条件ごとに結合フラグメントの内部を点線で区切り、その区画を選ぶ条件(ガード)と処理を記述します。条件は[]で囲んで示します。
区間は、いくつに分けても構いませんが、条件を満たす最初の区間を実行します。いずれの条件も満たさない場合はどの区間も実行されません。必ずいずれかの区画を実行したい場合は、それ以外を表すelseの区間を用意します。

結合フラグメント

- opt

■ オプション処理を表すインタラクションオペレータ
■ 条件（ガード）が1つだけのaltと同じ

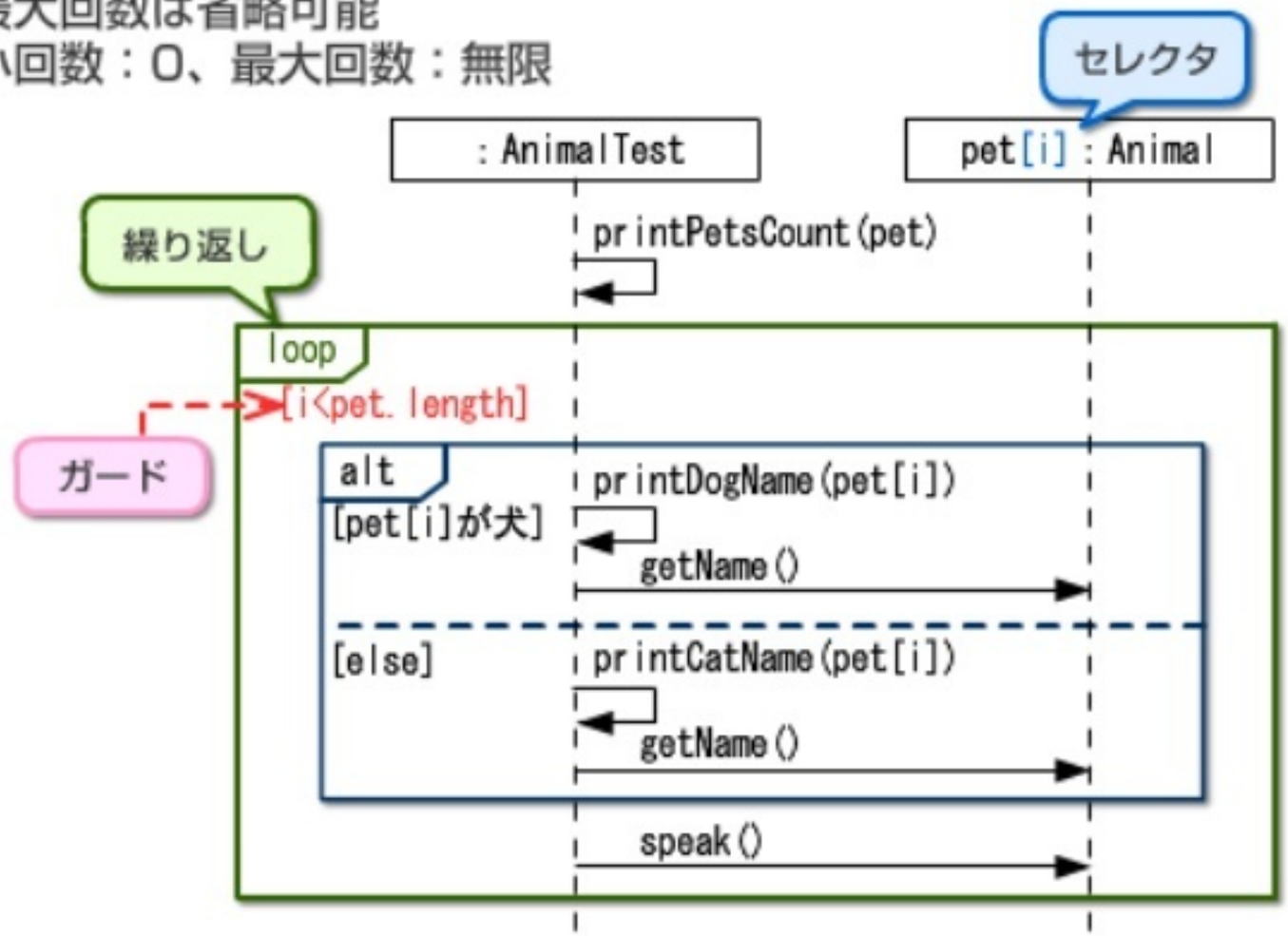


『opt』はオプション処理を示すインタラクションオペレータです。optはガードが1つしかないaltと同じです。optはガードを満たす場合に結合フラグメント内の処理を実行します。

結合フラグメント

● loop

■ 繰り返し処理を表すインタラクションオペレータ
■ 繰り返し回数は最小回数、最大回数、条件（ガード）で示す
■ loop(最小回数、最大回数)
　－最小回数、最大回数は省略可能
　－省略時の最小回数：0、最大回数：無限



次のページにソースを表示します。

『loop』は繰り返しを示すインタラクションオペレータです。繰り返しの回数は最小回数、最大回数、およびガードで示します。最小回数と最大回数はloopに続く( )に書きます。
loopは処理を最小回数繰り返した後、ガードを満たさなくなるか、最大回数になるまで処理を繰り返します。最小回数の省略値は0、最大回数の省略値は＊(無限)です。ガードを省略すると最大回数まで繰り返します。
この例のpetオブジェクトの[i]は『セレクタ』です。セレクタは同じ種類の複数のオブジェクトの中の1つを表す表記です。
「ソースを表示」ボタンをクリックし、シーケンス図とJavaコードとの対応を確認してください。

【AnimalTest.java】

```
1   import java.util.*;
2
3   class AnimalTest{
4       public static void main(String args[]){
5           // オブジェクトの生成と利用
6           // 生成したオブジェクトはスーパークラス Animal 型の配列で参照
7           Animal [] pet = new Animal[2];
8           pet[0] = new Dog("John");
9           pet[1] = new Cat("Tama");
10
11          //System.out.println("I have " + pet.length + " pets.");
12          printPetsCount(pet);
13          for(int i = 0 ; i<pet.length; i++){
14              // instanceof はオブジェクトのクラスを調べる演算子
15              if ( pet[i] instanceof Dog ){
16                  // pet[i]が Dog クラスのオブジェクトだった場合
17                  printDogName(pet[i]);
18              }else{
19                  // pet[i]が Dog クラスのオブジェクト以外だった場合
20                  printCatName(pet[i]);
21              }
22              System.out.println(pet[i].getName() + " Come on!!");
23              // speak メソッドをオーバーライドしているので、ポリモーフィズムが機能する
24              pet[i].speak();
25          }
26      }
27
28      private static void printPetsCount(Animal[] pet){
29          System.out.println("I have " +pet.length + " pets.");
30      }
31
32      private static void printDogName(Animal animal){
33          System.out.print(animal.getName() + " is dog. ");
34      }
35
36      private static void printCatName(Animal animal){
37          System.out.print(animal.getName() + " is cat. ");
38      }
39  }
40
```

```
41  abstract class Animal{
42      // 属性(メンバ変数)の定義
43      private String name;
44
45      // コンストラクタ
46      public Animal(String newName){
47          name = newName;
48      }
49
50      // 操作(メソッド)の定義
51      public String getName(){
52          return name;
53      }
54
55      // 抽象操作の定義
56      abstract public void speak();
57  }
58
59  class Dog extends Animal{
60      // 属性(メンバ変数)の定義
61      private int weight = 100;
62
63      // コンストラクタ
64      public Dog(String newName){
65          super(newName);
66      }
67
68      // 抽象操作の実装
69      public void  speak(){
70          System.out.println("Wow Wow , Wow Wow ");
71      }
72  }
73
74  class Cat extends Animal{
75      // コンストラクタ
76      public Cat(String newName){
77          super(newName);
78      }
79
80      // 抽象操作の実装
81      public void  speak(){
82          System.out.println("Nya---, Nya---");
83      }
84  }
85
```